

73. 矩阵置零

1. 问题

2. 方法

好的，我们来提供“矩阵置零” (LeetCode 73) 的两种解法：普通思路（使用额外空间）和优化思路（O(1) 空间）。

1. 问题理解

- **目标：** 给定一个 $m \times n$ 的矩阵 `matrix`。如果矩阵中某个元素 `matrix[i][j]` 为 0，则需要将该元素所在的**整行**和**整列**都置为 0。
- **输入：** 一个二维整数数组 `matrix`。
- **输出：** 无返回值，直接在**原地**修改输入的 `matrix`。
- **约束：** 鼓励使用原地算法，特别是 O(1) 额外空间的解法。

2. 普通思路（使用额外空间 O(m+n)）

- **思路：** 使用两个布尔数组分别记录需要置零的行和列。
- **步骤：**
 1. 创建 `rowZero` (大小 m) 和 `colZero` (大小 n) 两个布尔数组，初始化为 `false`。
 2. 遍历矩阵，如果 `matrix[i][j] == 0`，则设置 `rowZero[i] = true` 和 `colZero[j] = true`。
 3. 再次遍历矩阵，如果 `rowZero[i]` 或 `colZero[j]` 为 `true`，则将 `matrix[i][j]` 置为 0。

示例演示（普通思路）

`matrix = [[1, 1, 1], [1, 0, 1], [1, 1, 1]]` ($m=3, n=3$)

1. **初始化：** `rowZero = [false, false, false]` , `colZero = [false, false, false]`。
2. **第一次遍历（标记）：**

- 遍历到 `matrix[1][1] == 0`。
- 设置 `rowZero[1] = true`，`colZero[1] = true`。
- 标记结果：`rowZero = [false, true, false]`，`colZero = [false, true, false]`。

3. 第二次遍历（置零）：

- `i=0, j=0`：`rowZero[0]=false`，`colZero[0]=false` → `matrix[0][0]` 不变 (1)。
- `i=0, j=1`：`rowZero[0]=false`，`colZero[1]=true` → `matrix[0][1]` 置为 0。
- `i=0, j=2`：`rowZero[0]=false`，`colZero[2]=false` → `matrix[0][2]` 不变 (1)。
- `i=1, j=0`：`rowZero[1]=true`，`colZero[0]=false` → `matrix[1][0]` 置为 0。
- `i=1, j=1`：`rowZero[1]=true`，`colZero[1]=true` → `matrix[1][1]` 置为 0。
- `i=1, j=2`：`rowZero[1]=true`，`colZero[2]=false` → `matrix[1][2]` 置为 0。
- `i=2, j=0`：`rowZero[2]=false`，`colZero[0]=false` → `matrix[2][0]` 不变 (1)。
- `i=2, j=1`：`rowZero[2]=false`，`colZero[1]=true` → `matrix[2][1]` 置为 0。
- `i=2, j=2`：`rowZero[2]=false`，`colZero[2]=false` → `matrix[2][2]` 不变 (1)。

4. 完成！ 最终结果 `[[1, 0, 1], [0, 0, 0], [1, 0, 1]]`。

代码实现（普通思路 $O(m+n)$ 空间）

```
import java.util.HashSet;
import java.util.Set;

class Solution_O_m_plus_n_Space {
    public void setZeroes(int[][] matrix) {
        if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
            return;
        }
        int m = matrix.length;
        int n = matrix[0].length;

        boolean[] rowZero = new boolean[m];
        boolean[] colZero = new boolean[n];

        // 1. 第一次遍历：标记需要置零的行和列
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
```

```

        if (matrix[i][j] == 0) {
            rowZero[i] = true;
            colZero[j] = true;
        }
    }
}

// 2. 第二次遍历：根据标记进行置零
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        if (rowZero[i] || colZero[j]) {
            matrix[i][j] = 0;
        }
    }
}
}
}

```

3. 更好的方法（使用第一行/第一列作为标记 - O(1) 额外空间）

- **思路：** 利用矩阵的第一行和第一列来存储标记信息，并用两个额外变量记录原始第一行/列的状态。
- **步骤：**
 1. 用 `isFirstRowZero` 和 `isFirstColZero` 记录原始第一行/列是否有 0。
 2. 遍历矩阵内部（非第一行/列），如果 `matrix[i][j] == 0`，则标记 `matrix[i][0] = 0` 和 `matrix[0][j] = 0`。
 3. 再次遍历矩阵内部，根据 `matrix[i][0]` 或 `matrix[0][j]` 是否为 0，将 `matrix[i][j]` 置零。
 4. 最后根据 `isFirstRowZero` 和 `isFirstColZero` 将第一行/列置零。

示例演示（O(1) 空间法）

`matrix = [[0, 1, 2, 0], [3, 4, 5, 2], [1, 3, 1, 5]]` (m=3, n=4)

1. **检查第一行/列状态：** `isFirstRowZero = true` , `isFirstColZero = true` 。
2. **使用第一行/列记录内部信息：** 遍历内部，无 0。第一行/列标记不变。
3. **根据标记置零内部元素：**

- `matrix[1][3]` 因 `matrix[0][3]==0` 置为 0。
- `matrix[2][3]` 因 `matrix[0][3]==0` 置为 0。
- 矩阵变为 `[[0, 1, 2, 0], [3, 4, 5, 0], [1, 3, 1, 0]]`。

4. 根据标志置零第一行/列:

- 因 `isFirstRowZero=true`，第一行置零 → `[[0, 0, 0, 0], [3, 4, 5, 0], [1, 3, 1, 0]]`。
- 因 `isFirstColZero=true`，第一列置零 → `[[0, 0, 0, 0], [0, 4, 5, 0], [0, 3, 1, 0]]`。

5. 完成! 最终结果 `[[0, 0, 0, 0], [0, 4, 5, 0], [0, 3, 1, 0]]`。

代码实现 (O(1) 空间法)

```
/*
 * @lc app=leetcode.cn id=73 lang=java
 * @lcpr version=30204
 *
 * [73] 矩阵置零
 */

// @lcpr-template-start

// @lcpr-template-end
// @lc code=start
class Solution { // O(1) space solution
    public void setZeroes(int[][] matrix) {
        if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
            return;
        }
        int m = matrix.length;
        int n = matrix[0].length;

        boolean isFirstRowZero = false;
        boolean isFirstColZero = false;

        // 1. 检查第一行是否包含 0
        for (int j = 0; j < n; j++) {
            if (matrix[0][j] == 0) {
```

```

        isFirstRowZero = true;
        break;
    }
}

// 1. 检查第一列是否包含 0
for (int i = 0; i < m; i++) {
    if (matrix[i][0] == 0) {
        isFirstColZero = true;
        break;
    }
}

// 2. 使用第一行/列记录内部元素的零信息
// 注意：从 i=1, j=1 开始，避免干扰第一行/列的原始状态判断
for (int i = 1; i < m; i++) {
    for (int j = 1; j < n; j++) {
        if (matrix[i][j] == 0) {
            matrix[i][0] = 0;
            matrix[0][j] = 0;
        }
    }
}

// 3. 根据第一行/列的标记置零内部元素
// 注意：从 i=1, j=1 开始
for (int i = 1; i < m; i++) {
    for (int j = 1; j < n; j++) {
        if (matrix[i][0] == 0 || matrix[0][j] == 0) {
            matrix[i][j] = 0;
        }
    }
}

// 4. 根据标志置零第一行
if (isFirstRowZero) {
    for (int j = 0; j < n; j++) {
        matrix[0][j] = 0;
    }
}

```

```

    }
}

// 4. 根据标志置零第一列
if (isFirstColZero) {
    for (int i = 0; i < m; i++) {
        matrix[i][0] = 0;
    }
}
}
// @lc code=end

/*
// @lcpr case=start
// [[1,1,1],[1,0,1],[1,1,1]]\\n
// @lcpr case=end

// @lcpr case=start
// [[0,1,2,0],[3,4,5,2],[1,3,1,5]]\\n
// @lcpr case=end

*/

// You can rename the O(m+n) space solution class if needed,
// e.g., Solution_ExtraSpace, and keep the O(1) space solution as the main
'Solution' class
// for LeetCode submission.

```

4. 如何在面试中讲解

1. **确认问题：** "目标是在原地将矩阵中 0 元素所在的行和列都置零。"
2. **提出 $O(m+n)$ 空间解法：** "一个直接的方法是用两个额外的布尔数组分别记录需要置零的行和列，然后根据这两个数组更新原矩阵。这需要 $O(m+n)$ 的额外空间。" (可以展示或描述 `Solution_O_m_plus_n_Space` 的代码思路)

3. **引入 O(1) 空间解法：** "为了优化空间，我们可以利用矩阵本身的第一行和第一列作为标记。 `matrix[i][0]` 标记第 i 行， `matrix[0][j]` 标记第 j 列。 "
4. **解释关键点（冲突处理）：** "因为 `matrix[0][0]` 同时属于第一行和第一列，我们需要两个额外的布尔变量来单独记录原始第一行和第一列是否需要置零。 "
5. **分步讲解 O(1) 算法：** (按照 O(1) 代码的 4 个步骤讲解)
- "第一步，检查原始的第一行和第一列，设置 `isFirstRowZero` 和 `isFirstColZero` 标志。 "
 - "第二步，遍历矩阵内部（除去第一行第一列），如果 `matrix[i][j]` 为 0，就在对应的 `matrix[i][0]` 和 `matrix[0][j]` 上做标记（置为 0）。 "
 - "第三步，再次遍历矩阵内部，根据 `matrix[i][0]` 和 `matrix[0][j]` 的标记，将内部元素 `matrix[i][j]` 置零。 "
 - "第四步，最后根据 `isFirstRowZero` 和 `isFirstColZero` 标志，将第一行和第一列整体置零（如果需要的话）。 "
6. **复杂度：** "O(m+n)空间解法时间 O(mn)，空间 O(m+n)。O(1)空间解法时间 O(mn)，空间 O(1)。 "

找到具有 1 个许可证类型的类似代码